


```
import os

# This creates all the required folders automatically inside 'content'
os.makedirs('/content/dataset/train/Fruits', exist_ok=True)
os.makedirs('/content/dataset/train/Vegetables', exist_ok=True)
os.makedirs('/content/dataset/train/Dairy', exist_ok=True)
os.makedirs('/content/dataset/test', exist_ok=True)

print("All folders created successfully!")
```

All folders created successfully!

```
from google.colab import files
import os

# Function to upload files directly into a specific folder
def upload_to_folder(folder_path):
    print(f"\n--- Select images for: {os.path.basename(folder_path)} ---")
    uploaded = files.upload()
    for filename in uploaded.keys():
        os.rename(filename, os.path.join(folder_path, filename))
    print(f"Done uploading to {folder_path}!")

# Upload training images
upload_to_folder('/content/dataset/train/Fruits')
upload_to_folder('/content/dataset/train/Vegetables')
upload_to_folder('/content/dataset/train/Dairy')

# Upload test images
upload_to_folder('/content/dataset/test')
```

--- Select images for: Fruits ---

No files selected.

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_3455/1793723808.py in <cell line: 0>()
     11
     12 # Upload training images
--> 13 upload_to_folder('/content/dataset/train/Fruits')
     14 upload_to_folder('/content/dataset/train/Vegetables')
     15 upload_to_folder('/content/dataset/train/Dairy')
```

4 frames

```
/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in
read_reply_from_input(message_id, timeout_sec)
     94     reply = _read_next_input_message()
     95     if reply == _NOT_READY or not isinstance(reply, dict):
--> 96         time.sleep(0.025)
```

```
---/ 96         time.sleep(0.025)
      97         continue
      98     if (
```

```
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from PIL import Image

# 1. SETUP PATHS AND PARAMETERS
TRAIN_DIR = '/content/dataset/train'
TEST_DIR = '/content/dataset/test'
IMG_SIZE = (150, 150)
BATCH_SIZE = 4
CLASSES = ['Dairy', 'Fruits', 'Vegetables']

# 2. DATA PREPROCESSING
train_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)

# 3. BUILD THE CLASSIFIER MODEL
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 4. TRAIN THE MODEL
print("Training the AI model...")
model.fit(train_generator, epochs=10)

# 5. PREDICT ON UNSEEN TEST IMAGES
```

```

def predict_image(image_path):
    img = Image.open(image_path).resize(IMG_SIZE)
    img_array = np.array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    predictions = model.predict(img_array)
    predicted_class = CLASSES[np.argmax(predictions)]
    confidence = np.max(predictions) * 100

    print(f"Image: {os.path.basename(image_path)} | Result: {predicted_class}

# 6. RUN PREDICTIONS
print("\n--- Test Results ---")
test_images = [f for f in os.listdir(TEST_DIR) if f.lower().endswith(('.jpg'

for test_img in test_images:
    full_path = os.path.join(TEST_DIR, test_img)
    predict_image(full_path)

```

```

Found 8 images belonging to 3 classes.
Training the AI model...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_c
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
2/2 ━━━━━━━━━━━ 2s 189ms/step - accuracy: 0.3750 - loss: 8.9010
Epoch 2/10
2/2 ━━━━━━━━━━━ 0s 169ms/step - accuracy: 0.3750 - loss: 3.2403
Epoch 3/10
2/2 ━━━━━━━━━━━ 0s 160ms/step - accuracy: 0.2500 - loss: 2.0420
Epoch 4/10
2/2 ━━━━━━━━━━━ 0s 157ms/step - accuracy: 0.5000 - loss: 0.9513
Epoch 5/10
2/2 ━━━━━━━━━━━ 0s 160ms/step - accuracy: 0.6250 - loss: 0.8457
Epoch 6/10
2/2 ━━━━━━━━━━━ 0s 166ms/step - accuracy: 0.7500 - loss: 0.6999
Epoch 7/10
2/2 ━━━━━━━━━━━ 0s 177ms/step - accuracy: 1.0000 - loss: 0.4829
Epoch 8/10
2/2 ━━━━━━━━━━━ 0s 159ms/step - accuracy: 1.0000 - loss: 0.3394
Epoch 9/10
2/2 ━━━━━━━━━━━ 0s 158ms/step - accuracy: 0.8750 - loss: 0.2321
Epoch 10/10
2/2 ━━━━━━━━━━━ 0s 169ms/step - accuracy: 1.0000 - loss: 0.1256

--- Test Results ---
1/1 ━━━━━━━━━━━ 0s 142ms/step
Image: broccoli-1.jpeg | Result: Vegetables (91.14%)
1/1 ━━━━━━━━━━━ 0s 65ms/step
Image: 360_F_1876743781_JUrRnzGcY1hkW2ssZDdPNjh2PZ7h6Pns.jpg | Result: Vegeta

```

